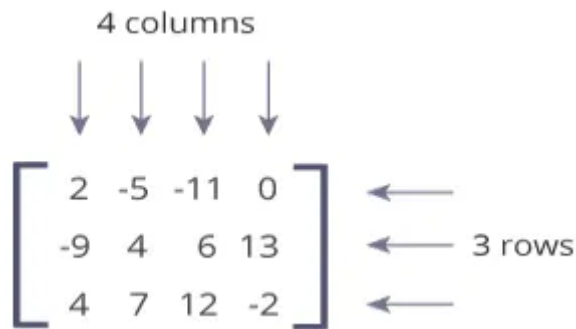


Python Matriks dan NumPy Array

Matriks adalah struktur data dua dimensi di mana angka-angka disusun menjadi baris dan kolom. Contohnya :



Matriks ini adalah matriks 3x4 (dibaca "tiga kali empat") karena memiliki 3 baris dan 4 kolom.

Python Matriks

Python tidak memiliki tipe built-in untuk matriks. Namun, kita dapat memperlakukan list di dalam list sebagai matriks. Sebagai contoh:

```
In [1]: A = [[1, 4, 5],
             [-5, 8, 9]]
```

Kita dapat memperlakukan list ini sebagai matriks yang memiliki 2 baris dan 3 kolom.



Contoh Program

```
In [2]: A = [[1, 4, 5, 12],
             [-5, 8, 9, 0],
             [-6, 7, 11, 19]]
```

Untuk mencetak atau mendapatkan seluruh nilai yang ada pada matriks di atas dapat digunakan perintah :

```
In [3]: print("A =", A)
```

```
A = [[1, 4, 5, 12], [-5, 8, 9, 0], [-6, 7, 11, 19]]
```

Selain itu, kita juga dapat mendapatkan elemen tertentu dari matriks tersebut dengan cara :

```
In [4]: print("A[1] =", A[1])      # Baris ke-2

        print("A[1][2] =", A[1][2])  # Elemen ke-3 dari baris ke-2

        print("A[0][-1] =", A[0][-1])  # Elemen terakhir dari Baris ke-1
```

```
A[1] = [-5, 8, 9, 0]
```

```
A[1][2] = 9
```

```
A[0][-1] = 12
```

NumPy Array

NumPy adalah sebuah library untuk komputasi ilmiah yang memiliki dukungan untuk objek array berdimensi-N. NumPy menyediakan deretan angka multidimensi (yang sebenarnya adalah sebuah objek). Contohnya:

```
In [5]: import numpy as np

        a = np.array([1, 2, 3])

        print(a)

        print(type(a))
```

```
[1 2 3]
```

```
<class 'numpy.ndarray'>
```

Seperti yang Anda lihat, kelas array NumPy disebut ndarray.

Bagaimana cara membuat array NumPy?

Ada beberapa cara untuk membuat array NumPy.

1. Array bilangan bulat, float dan Bilangan kompleks

```
In [6]: A = np.array([[1, 2, 3], [3, 4, 5]])  # Bilangan bulat
        print(A)
```

```
[[1 2 3]
```

```
 [3 4 5]]
```

```
In [7]: A = np.array([[1.1, 2, 3], [3, 4, 5]])  # float
        print(A)
```

```
[[1.1 2.  3. ]
```

```
 [3.  4.  5. ]]
```

```
In [8]: A = np.array([[1, 2, 3], [3, 4, 5]], dtype=complex)  # bilangan kompleks
        print(A)
```

```
[[1.+0.j 2.+0.j 3.+0.j]
```

```
 [3.+0.j 4.+0.j 5.+0.j]]
```

2. Array angka nol dan satu

```
In [9]: zeroes_array = np.zeros((2, 3))
print(zeroes_array)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

Selain itu, kita juga dapat menetapkan `dtype` menjadi 32 bit (4 byte). Karenanya, array ini dapat mengambil nilai dari -2^{-31} hingga $2^{-31}-1$.

```
In [10]: ones_array = np.ones((1, 5), dtype=np.int32) # menentukan dtype
print(ones_array)
```

```
[[1 1 1 1 1]]
```

Operasi Matriks

Pada contoh program selanjutnya akan dibuat 3 jenis program yaitu : penjumlahan dua buah matriks, perkalian dua buah matriks dan transpos sebuah matriks. Jika menggunakan program python biasa, kita memerlukan `nested lists` untuk membuat program tersebut. Namun ada cara yang lebih baik yaitu menggunakan library NumPy.

Penambahan Dua Matriks

Addition of Matrices

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$A + B = \begin{bmatrix} a_{11}+b_{11} & a_{12}+b_{12} \\ a_{21}+b_{21} & a_{22}+b_{22} \end{bmatrix}$$

Kita dapat menggunakan operator `+` untuk menambahkan elemen yang sesuai dari dua matriks NumPy.

```
In [11]: A = np.array([
    [3, 4],
    [2, 1],
    ]);

B = np.array([
    [1, 5],
    [3, 7],
    ]);

C = A + B;

print(C);
```

```
[[4 9]
 [5 8]]
```

Perkalian Dua Matriks

$$\begin{bmatrix} 4 & 5 \\ 2 & 6 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$= \begin{bmatrix} (4 \times 1) + (5 \times 3) & (4 \times 2) + (5 \times 4) \\ (2 \times 1) + (6 \times 3) & (2 \times 2) + (6 \times 4) \end{bmatrix}$$

$$= \begin{bmatrix} 4 + 15 & 8 + 20 \end{bmatrix}$$

$$\begin{matrix} 12 + 18 & 4 + 24 \\ = \begin{bmatrix} 19 & 28 \\ 20 & 28 \end{bmatrix} \end{matrix}$$

Untuk mengalikan dua matriks, kita menggunakan metode `dot()`.

Catatan: * digunakan untuk perkalian array (perkalian elemen yang sesuai dari dua array) bukan perkalian matriks.

```
In [12]: A = np.array(
          [[3, 4],
          [2, 1]])

          B = np.array(
          [[1, 5],
          [3, 7]])

          C = A.dot(B)

          print(C)
```

```
[[15 43]
 [ 5 17]]
```

Transpos Matriks

$$A = \begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} a & c & e \\ b & d & f \end{bmatrix}$$

Kita dapat menggunakan `numpy.transpose` untuk menghitung transpos matriks.

```
In [13]: A = np.array(
          [[3, 4],
          [2, 1]])

          print(A.transpose())
```

```
[[3 2]
 [4 1]]
```

Tugas

Pada contoh program di atas, sudah di buat 3 program dengan memanfaatkan library NumPy yaitu :

1. Penambahan dua matriks
2. Perkalian dua matriks
3. Transpos matriks

Tugas anda adalah membuat 3 program tersebut tanpa menggunakan library NumPy.

1. Penambahan dua matriks

```
In [14]: mat1 = [  
    [16, 4],  
    [20, 28],  
    ]  
  
    mat2 = [  
    [24, 28],  
    [32, 36],  
    ]  
  
    for x in range(0, len(mat1)):  
        for y in range(0, len(mat1[0])):  
            print(mat1[x][y] + mat2[x][y], end=' '),  
            print()
```

```
40 32  
52 64
```

2. Perkalian dua matriks

```
In [15]: mat1 = [  
    [16, 4],  
    [20, 28],  
    ]  
  
    mat2 = [  
    [24, 28],  
    [32, 36],  
    ]  
  
    mat3 = []  
  
    for x in range(0, len(mat1)):  
        row = []  
        for y in range(0, len(mat1[0])):  
            total = 0  
            for z in range(0, len(mat1)):  
                total = total + (mat1[x][z] * mat2[z][y])  
            row.append(total)  
        mat3.append(row)  
  
    for x in range(0, len(mat3)):  
        for y in range(0, len(mat3[0])):  
            print(mat3[x][y], end=' '),  
            print()
```

```
512 592  
1376 1568
```

3. Transpos matriks

```
In [16]: A = [[6, 10],  
    [14, 16],]
```

```
result = [[0, 0],  
          [0, 0]]  
  
# iterate through rows  
for i in range(len(A)):  
    # iterate through columns  
    for j in range(len(A[0])):  
        result[j][i] = A[i][j]  
  
for r in result:  
    print(r)
```

[6, 14]

[10, 16]